



NSMA - An efficient hybrid NSGA-II and Local search algorithm for a multi-objective Hub Location Problem

Optimizing Thailand railway transport infrastructure, for the new high-speed rail (HSR) line along the Northern Line

Le Tang Phu Quy^{1,2} Sunarin Chanta²

¹Faculty of Computer Science
Vietnam - Korea University of Information and Communication Technology
University of Da Nang

²Faculty of Industrial Technology and Management
King Mongkut's University of Technology North Bangkok

August 20, 2025

Table of Contents

- 1 Introduction
- 2 Problem Definition
- 3 Proposed Algorithm
- 4 Experimental Results
- 5 Conclusion

Introduction

- Hub and Spoke network are widely used for transport, telecom and computers
- With *economies of scales*, costs are saved by transferring consolidated flows
- A *multi-objective* HLP is used to model the **railway** system
- Hub Location Problem (HLP) belongs to the **NP**-hard class
- New meta-heuristics algorithm is proposed to provide optimal trade-off solutions

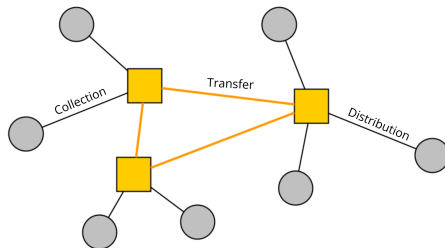


Figure: Hub and Spoke network

Related Works

- O'Kelly [6] proposed the first formulation for HLP. In 1994, Campbell [2] introduced p-hub center and hub covering problem.
- In reality, it is more beneficial to build **incomplete** hub network. Alumur [1] gives formulations for the single allocation hub location problems.
- **Multi-objective** models are used to solve real-world problems in transportation. Demir [4] proposes a bi-objective multiple allocation hub network design and routing problem and used NSGA-II algorithm for solution. For **railway** transportation planning, S. Chanta [7] presents a biobjective hub location model that considers congestion.
 - ▷ We formulate a tri-objective, uncapacitated, single allocation hub network design problem (USA HND).
 - ▷ Hybrid algorithm **NSMA** is proposed, combining *NSGA-II* and *Local search*.

Table of Contents

- 1 Introduction
- 2 Problem Definition**
- 3 Proposed Algorithm
- 4 Experimental Results
- 5 Conclusion

Input and Output

① Input:

- N : nodes, $H \subset N$: potential hubs
- Demand matrix w_{ij} : amount of passenger flow from i to j
- Distance (Cost) matrix C_{ij} , travel time $t_{ij} = C_{ij}$

② Assumption:

- Single Allocation: each node is allocated to exactly one hub
- Hub network is directed

③ Output: (decision variables)

x_{ij} : hub location and assignment of nodes to hubs

z_{ij} : hub network

Objectives

$$\text{Minimize } Z_1 = \sum_{i \in H} \sum_{j \in H} FL_{ij} z_{ij} + \sum_{k \in H} FH_k x_{kk} + \sum_{i \in N} (\chi C_{ik} O_i + \delta C_{ki} D_i) x_i x_i + \sum_{i \in H} \sum_{j \in H} \sum_{k \in N} \alpha C_{ij} f_{ij}^k$$

$$\text{Minimize } Z_2 = \max_{i,j \in N} \{t_i x_i + \beta s_{x_i x_j} + t_{x_j j}\}$$

$$\text{Maximize } Z_3 = \sum_{i \in N} O_i [C_i x_i \leq L]$$

(note: $[B] = 1$ if B is true, 0 otherwise. It is a binary variable)

Z_1 is the total cost, including fixed costs and transportation costs.

Z_2 is the maximum travel time of an O-D pair (i, j) .

Z_3 is the amount of demand covered by the system.

Characteristics of the model

- The model has $O(n^3)$ constraints and variables (abstracted)
- Under single objective: *Mixed Integer Programming* model.
- This model is an adaptation of the work by Alumur [1] and extension of S. Chanta's [7]
- Congestion at hub (delay time t_i [7] at stations) are considered in travel time Z_2

Table of Contents

- 1 Introduction
- 2 Problem Definition
- 3 Proposed Algorithm**
- 4 Experimental Results
- 5 Conclusion

NSGA-II

Our algorithm is built on NSGA-II [3]:

- It is one of the most well-known multi-objective evolutionary algorithms (MOEAs)
- **Elitist:** NSGA-II combines parents and offspring, selecting the best for survival.
- Features:

Fast non-dominated sorting

Crowding distance for maintaining **diversity** of the population

Binary tournament is used for Selection; and the proposed nondominated elitist selection process (`nsga_elitist()`) is used for *replacement*.

▷ Solution representation, crossover and mutation operators need to be *designed* for specific application.

General outline of the proposed algorithm

Memetic Algorithm = Genetic Algorithm + Local Search

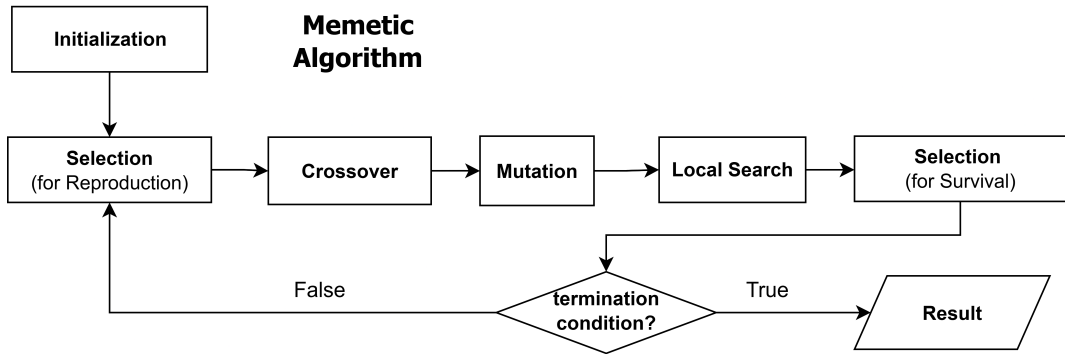


Figure: Main steps in a Memetic Algorithm

Representation of solution - Part I: Hub location

This work uses *allocation vector* representation.

- $A_i = k$ ($k \neq i$) if node i is allocated to hub k ; otherwise, $A_k = k$ if k is a hub
- Node can only be allocated to hub (i.e. $A_{A_i} = A_i$)

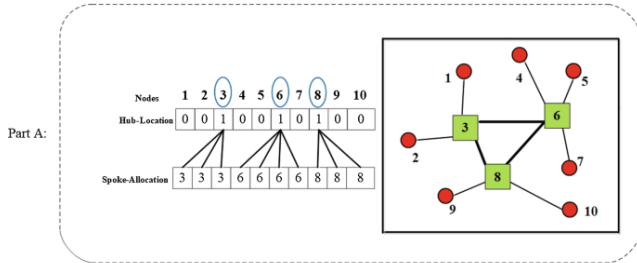


Figure: Allocation vector representation, image source: [9]

Solution representation - Part II : hub network (arc location)

Representation – binary string:

- Denote $\mathcal{H}(A) = \{h_1, h_2, \dots, h_p\}$ the hubs in the representation A , then $p = |\mathcal{H}|$ is the number of hubs.
- We use a binary string of length p^2 bits where $(i - 1)p + j$ -th bit corresponds to the arc (h_i, h_j) .
- Validation conditions: (1) No self-loops and multiple edges allowed. (2) The hub network must be *strongly connected*

Evaluation of objective values:

We assume the representation is valid, if not, apply the `repair()` method

- Compute all-pair shortest paths using Floyd algorithm [5] in $O(n^3)$
- Loop over every O-D pair (i, j) and compute Z_1, Z_2, Z_3 in $O(n^2)$
- Total time complexity for evaluation is $O(n^3)$

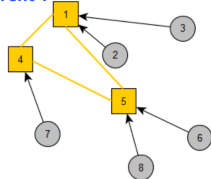
Crossover operator - Phase I (Hub Crossover)

The steps of the crossover operator on parents (P_1, P_2) to create children (C_1, C_2) is as follows:

- 1 **Hub exchange:** randomly distribute $\mathcal{H}(P_1) \cup \mathcal{H}(P_2)$ to $\mathcal{H}(C_1)$ and $\mathcal{H}(C_2)$ with uniform probability. The common hubs $\mathcal{H}(P_1) \cap \mathcal{H}(P_2)$ are inherited in both C_1, C_2
- 2 **Hub mapping:** create mappings $\mathcal{M}_{C_1}, \mathcal{M}_{C_2}$ for C_1, C_2 respectively. $\mathcal{M}_A(k) = k$ if $k \in \mathcal{H}(A)$, else $\mathcal{M}_A(h) = \arg \min_{k \in \mathcal{H}(A)} \text{distance}(k, h)$. If a hub h is not present in $\mathcal{H}$, then it is mapped to the nearest hub $k \in \mathcal{H}$.
- 3 **Crossover:** $C_1 \leftarrow P_1; C_2 \leftarrow P_2$, then 1-point crossover is applied on the allocation vectors C_1, C_2 .
- 4 **Repair:** C_1 and C_2 's allocations are fixed according to $\mathcal{M}_{C_1}$ and $\mathcal{M}_{C_2}$, ensuring nodes are assigned to proper hub.

Example - phase I of crossover operator

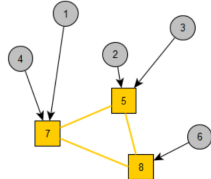
Parent 1



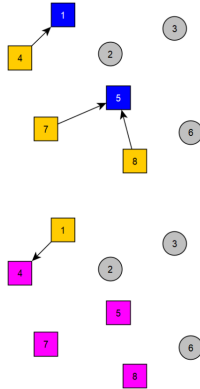
1	2	3	4	5	6	7	8
1	1	1	4	5	5	4	5

1	2	3	4	5	6	7	8
7	5	5	7	5	8	7	8

Parent 2



Hub exchange & mapping



Allocation crossover

	1	2	3	4	5	6	7	8
c1	1	1	1	4	5	8	7	8

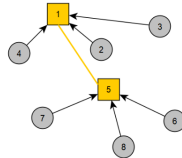
	1	2	3	4	5	6	7	8
c2	7	5	5	7	5	5	4	5

repair

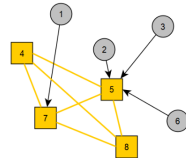
	1	2	3	4	5	6	7	8
c1	1	1	1	1	5	5	5	5

	1	2	3	4	5	6	7	8
c2	7	5	5	4	5	5	7	8

Child 1



Child 2



Crossover - Phase II (Arcs Crossover)

Denote $\mathcal{A}(A)$ the arcs in the representation A . In a similar manner, the second part of representation is crossed over as described below:

- ① **Arc exchange:** randomly distribute $\mathcal{A}(P_1) \cup \mathcal{A}(P_2)$ to $\mathcal{A}(C_1)$ and $\mathcal{A}(C_2)$ with ratio $|\mathcal{H}(C_1)|^2/|\mathcal{H}(C_2)|^2$. The common arcs $\mathcal{A}(P_1) \cap \mathcal{A}(P_2)$ are inherited in both C_1, C_2 .
- ② **Arc mapping:** each arc (u, v) is mapped to $(\mathcal{M}_C(u), \mathcal{M}_C(v))$. Invalid self-loops and multiple edges are removed.
- ③ **Repair:** by fixing the strong connectivity property (the `repair()` method).

Mutation operators

The four mutation operators below has the same probability to be chosen ($\frac{1}{4}$):

- **m1 - Make hub**: select a random node i and make it a hub
- **m2 - Erase hub**: select a random hub k and make it a node. All node j currently allocated to k is then assigned to a random hub (other than k).
- **m3 - Hub exchange**: exchange a hub k with a non-hub node i . All nodes currently assigned to k will be allocated to i .
- **m4 - Arc flip**: selects $\max\{2, \frac{|A|}{p}\}$ random bits in the arc representation and flips them. If the flipped bit is 1, then the corresponding arc is removed, otherwise it is added.

Mutation operators illustration

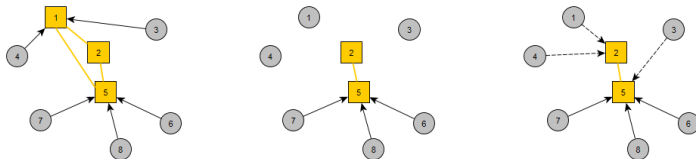


Figure: m2 erases hub 1

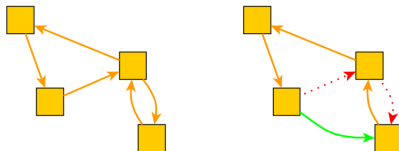


Figure: m4 flips 3 bits, adding 1 arcs and remove 2 arcs

Mutation operators illustration (cont'd)

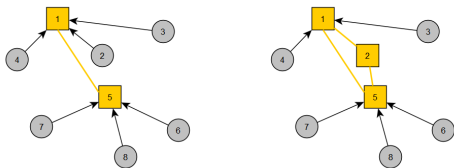


Figure: m1 makes new hub 2

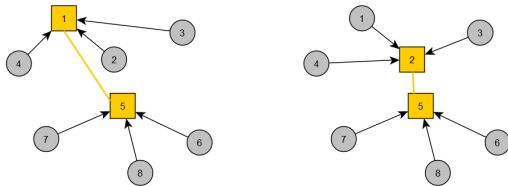


Figure: m3 exchanges hub 1 with node 2

Repair operator - Fix connectivity

Crossover and mutation (m4) operators may break the strong connectivity property of the hub network. To fix this, we apply the `repair()` method:

`repair()` method - fix strong connectivity

Denote $\bar{\mathcal{A}}$ the set of arcs **not** included in the representation A ;
and $a \rightsquigarrow b$ if node b is reachable from node a via the arcs $\mathcal{A}$

```
1: for  $k \in H : A_k = k$  do                                ▷ for each hub  $k$  established
2:   while not  $k \rightsquigarrow \mathcal{H} \setminus \{k\}$  do                ▷ there is node un-reachable from  $k$ 
3:      $(u, v) \leftarrow \arg \min_{i, j \in \mathcal{H}} \{C_{ij} \mid (i, j) \in \bar{\mathcal{A}} \mid k \rightsquigarrow i \wedge \neg k \rightsquigarrow j\}$     ▷ min-cost arc
4:     Add the arc  $(u, v)$  to the representation  $A$ 
5:      $\bar{\mathcal{A}} \leftarrow \bar{\mathcal{A}} \setminus \{(u, v)\}$ 
6:     Perform Depth-first Search to update reachability status
7:   end while
8: end for
```

NSGA-II algorithm for HLP

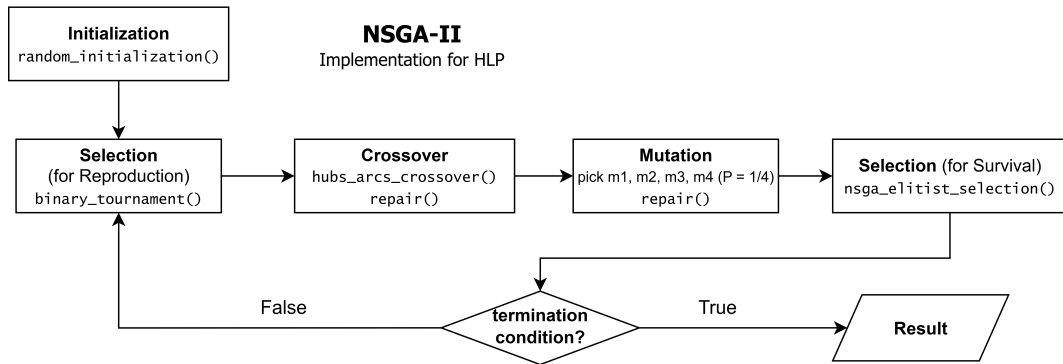


Figure: Detailed flow of NSGA-II algorithm for HLP

Local search - Neighborhood structure

Problem

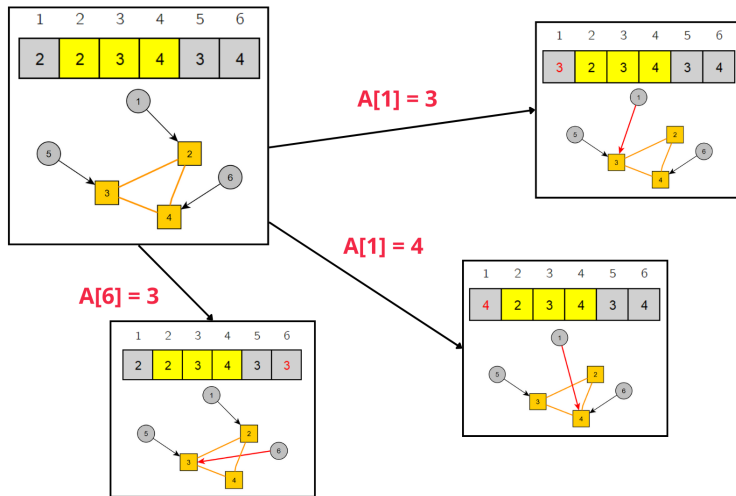
- The proposed crossover operator can preserve traits from both parents
- Mutation makes small changes to the hub location and network structure
- However, allocation pattern is not optimized, leading to sub-optimal solutions

▷ The solution is Local Search!

Neighborhood of A is the set of representation differs from A at exactly 1 position.

$$\mathcal{N}(A) = \{A' \mid \text{distance}(A, A') = 1\}$$

Neighborhood illustration



Apply local search in NSMA

$$\mathcal{N}(A) = \{A' \mid \text{distance}(A, A') = 1\}$$

- A representation A has exactly $(n - p)(p - 1)$ neighbors (p is the number of hubs)
- To optimize the allocation vector given a fixed hubs network, loops through all neighbors iteratively and select the *first* one that *dominates* current solution.
- Repeat the process until no more changes is found
- Local search is applied after every mutation and for every 25 generations
- **Problem:** However, for each neighbor, simply invoking `evaluate()` method will incur a time complexity of $O(n^3)$

Efficient local search implementation

Observation

Changing allocation A_i of node i from k to l only affect $O(n)$ components

- $C_{iA_i}(\chi O_i + \delta D_i) + \sum_j \alpha s C_{A_i A_j} w_{ij}$ (Z_1)
- $\{t_{iA_i} + \beta s_{A_i A_j} + t_{A_j j}\}$ (Z_2)
- $O_i[C_{iA_i} \leq L]$ (Z_3)

The implementation is then as follows:

- Compute changes to Z_1, Z_3 by subtracting and adding corresponding components
- For Z_2 , use data structures to maintain the set of travelling times efficiently

Evaluation cost for each neighbor is now $\mathcal{O}(n)$!

NSMA algorithm's detailed flow

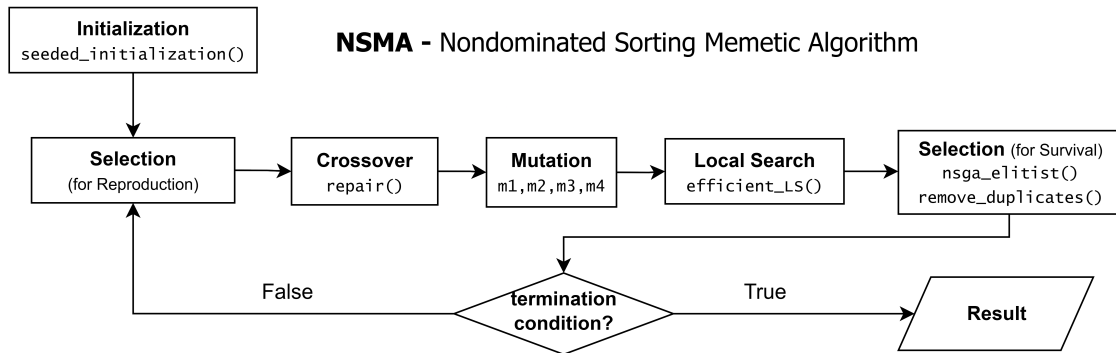


Figure: Detailed steps of NSMA algorithm

Table of Contents

- 1 Introduction
- 2 Problem Definition
- 3 Proposed Algorithm
- 4 Experimental Results**
- 5 Conclusion

Dataset for experiment

- The dataset is taken from [7], consists of 54 train stations along The Northern Line. The distance matrix is calculated using Haversine formula. The flow matrix contains the averaged amount of passenger from each origin station i to destination station j over a period of years.
- From this dataset, two problem instances with size $n = 10$ and $n = 54$ are derived for each variant. The parameter FL is only used in the Incomplete variant.

n	L	FH_k	FL_{ij}	Hub capacity	t_k
10	30	10^6	$10^5 \times C_{ij}$	Unlimited	10
54	60	10^7	$10^6 \times C_{ij}$	Unlimited	10

Table: Input parameters for instances

Experiment settings

- To have statistical significance and reliable measurement, each algorithm is run for 20 times on each instance
- Experiments are run on a laptop with CPU 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz with 16 GB RAM
- Algorithm parameters:

Parameter	NSGA-II	NSMA
POP_SIZE	500	500
NUM_GEN	200	200
p_c	0.95	0.95
p_m	0.3	0.24
$[f_{min}, f_{max}]$	$\{[5 \cdot 10^8, 10^{10}], [20, 800], [0, 93000]\}$	

Table: Algorithm parameters

Evaluation metrics

- ① Size of Pareto set
larger is better
Larger set indicates more diverse solutions
- ② Hyper Volume indicator (HV)
larger is better
HV indicates the objective space dominated by the points set
- ③ Generational Distance (GD)
smaller is better
how well the algorithm approximates to the references?
- ④ Inverted Generational Distance (IGD)
smaller is better
how well the reference set being approximated?
- ⑤ Spacing
smaller is better
measures the uniformity of distribution of points
computed by taking the standard deviation of the nearest-neighbor distances

Characteristics of metrics

Metrics	Requires reference set R	Requires reference point r	Goal	Time complexity
Size	No	No	Maximize	$\mathcal{O}(1)$
HV	No	Yes	Maximize	$\mathcal{O}(n^{d-1})$ [8]
GD	Yes	No	Minimize	$\mathcal{O}(n^2)$
IGD	Yes	No	Minimize	$\mathcal{O}(n^2)$
Spacing	No	No	Minimize	$\mathcal{O}(n^2)$

Table: An overview of used evaluation metrics

Settings for computation of metrics

- The true Pareto-front is unknown before-hand for large, complex real-world problem instances
- For each instance (each value of n), the reference set R_n is approximated by taking the non-dominated (best) solutions in the combined sets of solutions found by both algorithms in all 20 runs.
- To handle different objective value scales, we used min-max normalization to convert all coordinates to range $[0, 1]$ using formula:

$$x' = \frac{x - \min}{\max - \min}$$

Results

Result for Complete Hub network (Uncapacitated Single Allocation HLP)

n	size		HV		Spacing (10^{-2})		GD (10^{-3})		IGD (10^{-4})	
	NSGA	NSMA	NSGA	NSMA	NSGA	NSMA	NSGA	NSMA	NSGA	NSMA
10	17.4	19.1	0.801	0.804	16	16	1.41	0.805	156	109
54	271	343	0.927	0.931	2.45	2.04	1.64	0.729	9.94	7.91

Result for Incomplete Hub network variant

n	size		HV		Spacing (10^{-2})		GD (10^{-3})		IGD (10^{-4})	
	NSGA	NSMA	NSGA	NSMA	NSGA	NSMA	NSGA	NSMA	NSGA	NSMA
10	5.5	7.55	0	0	0.159	0.154	7.75	12.4	682	389
54	431	464	0.945	0.955	2.07	1.68	1.16	0.452	7.35	5.39

Table: Evaluation metrics of NSGA-II and NSMA on two problem variants

Results analysis

- NSMA's metrics have lower standard deviations than NSGA's
- NSMA has higher HV than NSGA-II on average
- NSMA generates more non-dominated points, distribute them evenly and has better convergence towards *optimal* Pareto front
- The result demonstrated the efficiency of Local Search in improving searching capability of the algorithm.

Pareto front visualization

Trade-off solutions found by the algorithm

ID	Z_1	Z_2	Z_3	Opened hubs
1	6.26×10^7	1.01×10^3	2.55×10^4	15
2	1.84×10^8	9.07×10^2	4×10^4	12 23
3	4.62×10^8	6.59×10^2	7.67×10^4	11 25 33
4	7.35×10^8	4.85×10^2	8.34×10^4	11 25 33 43
5	1.34×10^9	6.07×10^2	9.05×10^4	11 24 27 35 44 48 52
6	2.76×10^9	3.60×10^2	9.05×10^4	10 12 14 27 35 45 47 52 53

Table: Some Pareto solutions found by NSMA for Incomplete variant

► As more costs being invested to open hub facilities and building hub network (Z_1), the service quality (Z_2) and customer's willingness to use the service (Z_3) increases. Decision makers can make choices based on their preferences.

Table of Contents

- 1 Introduction
- 2 Problem Definition
- 3 Proposed Algorithm
- 4 Experimental Results
- 5 Conclusion**

Conclusion

- This work has reviewed some of the recent development trends in the field.
- By developing a comprehensive mathematical formulation with practical considerations, we obtain a highly useful and realistic model. This model well reflects the real-world problem of railway infrastructure planning.
- Next, we are going to integrate hub capacity constraints into the model. There are multiple levels of hub capacities (small, medium, large). They are decision variables in the model

Bibliography I

- [1] Sibel A Alumur, Bahar Y Kara, and Oya E Karasan. “The design of single allocation incomplete hub networks”. In: *Transportation Research Part B: Methodological* 43.10 (2009), pp. 936–951.
- [2] James F. Campbell. “Integer programming formulations of discrete hub location problems”. In: *European Journal of Operational Research* 72.2 (1994), pp. 387–405. ISSN: 0377-2217. DOI: [https://doi.org/10.1016/0377-2217\(94\)90318-2](https://doi.org/10.1016/0377-2217(94)90318-2). URL: <https://www.sciencedirect.com/science/article/pii/0377221794903182>.
- [3] Kalyanmoy Deb et al. “A fast and elitist multiobjective genetic algorithm: NSGA-II”. In: *IEEE transactions on evolutionary computation* 6.2 (2002), pp. 182–197.

Bibliography II

- [4] İbrahim Demir, Fatma Corut Ergin, and Berna Kiraz. “A new model for the multi-objective multiple allocation hub network design and routing problem”. In: *IEEE Access* 7 (2019), pp. 90678–90689.
- [5] Robert W Floyd. “Algorithm 97: shortest path”. In: *Communications of the ACM* 5.6 (1962), pp. 345–345.
- [6] Morton E. O'Kelly. “A quadratic integer program for the location of interacting hub facilities”. In: *European Journal of Operational Research* 32.3 (1987), pp. 393–404. ISSN: 0377-2217. DOI: [https://doi.org/10.1016/S0377-2217\(87\)80007-3](https://doi.org/10.1016/S0377-2217(87)80007-3). URL: <https://www.sciencedirect.com/science/article/pii/S0377221787800073>.

Bibliography III

- [7] Chanta Sunarin et al. “A Biobjective Hub Location Model with Consideration of Congestion for Railway Transportation Planning”. In: *Journal of Advances in Information Technology* 15 (Jan. 2024), pp. 414–421. ISSN: 1798-2340. DOI: 10.12720/jait.15.3.414-421. URL: <https://cir.nii.ac.jp/crid/1870021790741118976>.
- [8] Lyndon While et al. “A faster algorithm for calculating hypervolume”. In: *IEEE transactions on evolutionary computation* 10.1 (2006), pp. 29–38.
- [9] Wenying Zhang, Xifu Wang, and Kai Yang. “Uncertain multi-objective optimization for the water–rail–road intermodal transport system with consideration of hub operation process using a memetic algorithm: W. Zhang et al.”. In: *Soft Computing* 24.5 (2020), pp. 3695–3709.